PLANT INFORMATION DASHBOARD

Set up the daily logsheet job

This guide is for the Windows PC that downloads new logsheet days and sends them to staging. Staging cannot reach logsheet production. This PC can.

1. **Download**
   New days from logsheet production.
2. **Store**
   Each new day in the database on this PC.
3. **Send**
   A JSON copy of each stored day to 172.28.92.57.
4. **Load**
   Staging writes those files into its database.

The job runs every day at 16:00. The Windows task is named **Plant Dashboard logsheet sync**. It cannot type a password. Logins that the task uses must already be saved on this PC.

Before you start

Sign in to the dedicated PC as the Windows user who will own the daily task. Stay on that same user for every step below. The task uses that user's SSH key.

You need	Why
This PC can open logsheet production and 172.28.92.57	It downloads sheets here, then copies them to the API host.
The logsheet username and password	They go in `Frontend\.env.sync`. They are not copied to staging.
The root password for 172.28.92.57	Used once, while installing the SSH key.
A database dump from a PC that already stores Bekasi logsheets	An empty database cannot store the sheets. The site, tanks, and form definitions have to be there first.

Install these programs

1. Git.
2. Node.js 20 or newer. In PowerShell, `node -v` should print `v20` or higher.
3. Docker Desktop. Leave it running.
4. The Windows OpenSSH Client. In PowerShell, `ssh -V` should print a version line.

1. Get the project

Open PowerShell. Clone the `sit` branch. Use a folder path with no surprise spaces if you can. The examples below use `C:\PlantDashboard`.

```
git clone -b sit `
  https://github.com/riandharmawan/plant-information-dashboard.git `
  C:\PlantDashboard
cd C:\PlantDashboard
cd Frontend
npm ci
cd ..\Backend
npm ci
```

> **You should see** both `npm ci` commands finish without an error. The daily job needs the Frontend packages to read Excel, and the Backend packages to write the database.

2. Start the local database

The job stores each day on this PC before it sends a copy. If the local store fails, that day is not sent.

1. In the repo root, copy the example env file.

```
cd C:\PlantDashboard
copy .env.dev.example .env.dev
```

> **You should see** a file named `.env.dev`. Keep the database password that is already in the example, unless this PC already uses a different one. Do not put the staging database password here.

2. Start only the database.

```
docker compose --env-file .env.dev -f docker-compose.dev.yml up -d pid-db
```

> **You should see** the container `pid-db` running. It listens on `127.0.0.1:5437`.

3. On a PC that already has the working database, create a dump from the repo root.

```
.\Backend\scripts\staging-pg-dump-local.ps1
```

> **You should see** `pid_staging.dump` in that repo root. Copy the file to the dedicated PC with a USB drive or a private share. Do not commit it.

4. On the dedicated PC, restore the dump into `pid-db`.

```
docker cp .\pid_staging.dump pid-db:/tmp/pid_staging.dump
docker exec pid-db pg_restore -U pid_user -d pid_db `
  --clean --if-exists --no-owner --no-acl `
  /tmp/pid_staging.dump
```

> **You should see** the command return to PowerShell. A few “does not exist” lines on a new database can be ignored.

5. Check that the form list is present.

```
docker exec pid-db psql -U pid_user -d pid_db -c "SELECT count(*) FROM
logsheet_definitions;"
```

> **You should see** a count greater than 0. A count of 0 means the restore did not land. Do not register the daily task yet.

3. Save the logsheet login

1. Copy the example.

```
cd C:\PlantDashboard\Frontend
copy .env.example .env.sync
```

2. Open `Frontend\.env.sync` and fill `LOGSHEET_IDENTIFIER` and `LOGSHEET_PASSWORD`. Leave the form ids already in the file. This file stays on this PC.

Do not commit `.env.sync`. Do not copy the logsheet password into the staging server.

4. Save the staging address

```
cd C:\PlantDashboard\Frontend
copy .env.staging-push.example .env.staging-push
```

The example already points at `root@172.28.92.57` and `/opt/plant-information-dashboard`. Leave those values unless the API host has changed. This file stays on this PC.

5. Create an SSH key the task can use

The daily task cannot type a password or a passphrase. The private key stays on this PC. Only the public key goes to the server.

1. Create the key. Press Enter at both passphrase questions so the passphrase is empty. If Windows asks to overwrite a key that was never installed on the server, type `y`.

   ```
   ssh-keygen -t ed25519 -f $env:USERPROFILE\.ssh\id_ed25519
   ```

   In PowerShell, `-N '""'` does not mean an empty passphrase. It locks the key, and the daily task cannot unlock it. Use the command above and press Enter.

2. Confirm the key is unlocked. Do not add `-P`. On this Windows OpenSSH, `ssh-keygen -y -P` prints `Too many arguments.`

   ```
   ssh-keygen -y -f $env:USERPROFILE\.ssh\id_ed25519
   ```

   You should see a line that starts with `ssh-ed25519`, with no passphrase question. If a passphrase question appears, press Ctrl+C and create the key again.

3. Install the public key. These are the only commands that should ask for the root password. The password will not appear as you type. Run them one at a time.

   ```
   ssh root@172.28.92.57 "mkdir -p ~/.ssh; chmod 700 ~/.ssh"
   Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | ssh root@172.28.92.57 "cat >>
   ~/.ssh/authorized_keys"
   ssh root@172.28.92.57 "chmod 600 ~/.ssh/authorized_keys"
   ```

   You should see a return to the `PS` prompt with no error after you type the password.

4. Prove the task can log in alone.

   ```
   ssh -o BatchMode=yes root@172.28.92.57
   ```

   You should see a Linux prompt and no password question. Type `exit`.

`Permission denied (publickey,password)` means the server does not have this public key yet. Repeat step 3. Do not register the daily task until step 4 opens a shell.

6. Confirm the load script is on the server

The API host loads each JSON file with a script in the git checkout. The running container already knows how to write the database. You do not rebuild the API for this.

```
ssh root@172.28.92.57
```

Then on the server:

```
cd /opt/plant-information-dashboard
git pull origin sit
ls Backend/scripts/apply-staging-inbox.sh
exit
```

> **You should see** `ls` print `Backend/scripts/apply-staging-inbox.sh`. If pull says the server has local edits, stop and ask before changing those files.

7. Register the daily task

Still in PowerShell, as the same Windows user:

```
cd C:\PlantDashboard
.\Frontend\scripts\install-logsheet-sync-task.ps1
```

> You should see `Registered 'Plant Dashboard logsheet sync' daily at 16:00.`

To use another time, pass it. For example, `.\Frontend\scripts\install-logsheet-sync-task.ps1 18:30`. If the PC is off at that time, the task starts when the PC is next on. Leave the PC signed in as this user. The task runs for that user.

8. Run it once and read the reports

The first run on a new PC only downloads days that are not already saved as workbooks on this PC. A fresh clone has no workbooks, so the first run takes today. To send older days that staging does not have yet, add a range. Use the last date already on staging as `--from`.

```
node C:\PlantDashboard\Frontend\scripts\sync-and-push-staging.mjs `
  --from 2026-10-01 --to 2026-10-29
```

Change those dates to the range you need. A later daily run does not need the dates. It continues after the newest workbook on this PC.

Open this file	A good result
`Docs\exports\logsheet-sync\sync-report.txt`	Status: OK for the download and the local store.
`Docs\exports\staging-outbox\push-report.txt`	Status: OK, Failed: 0, Left in outbox: 0.

> **You should see** the sent files listed by name. They move to `Docs\exports\staging-outbox\sent` on this PC. On the API host they move to `/opt/plant-information-dashboard/staging-inbox/applied`.

The last lines of the push report always mention where a failed file would remain. If **Left in outbox** is 0, nothing is still waiting.

Each form also writes its own report under `Docs\exports`, for example `Docs\exports\storage-tank-sounding\sync-report.txt`. History is appended to the matching `sync.log` and to `Docs\exports\staging-outbox\push-log.txt`.

After it is running

Run the whole job again

This downloads any days still missing, stores them, and sends whatever is still in the outbox.

```
Start-ScheduledTask -TaskName "Plant Dashboard logsheet sync"
```

The task runs in the background. Read the two report files when it finishes. To watch it in the window instead:

```
node C:\PlantDashboard\Frontend\scripts\sync-and-push-staging.mjs
```

Fill a date gap

```
node C:\PlantDashboard\Frontend\scripts\sync-and-push-staging.mjs --from 2026-10-28 --to 2026-10-29
```

Send again without downloading

Use this when the local store succeeded and only the copy or the staging load failed. Waiting files stay in `Docs\exports\staging-outbox`.

```
node C:\PlantDashboard\Frontend\scripts\push-staging.mjs
```

If the send fails

The report says	What to do
`Permission denied (publickey,password)`	The SSH test in section 5 is failing. Fix the key, then run `push-staging.mjs` again. Do not download the sheets again.
`apply-staging-inbox.sh: No such file or directory`	On the API host, run `git pull origin sit` and check that the script exists. Then run `push-staging.mjs` again.
Left in outbox is greater than 0	Those files were not loaded. Read the Failed lines in `push-report.txt`, fix that cause, and run `push-staging.mjs` again.